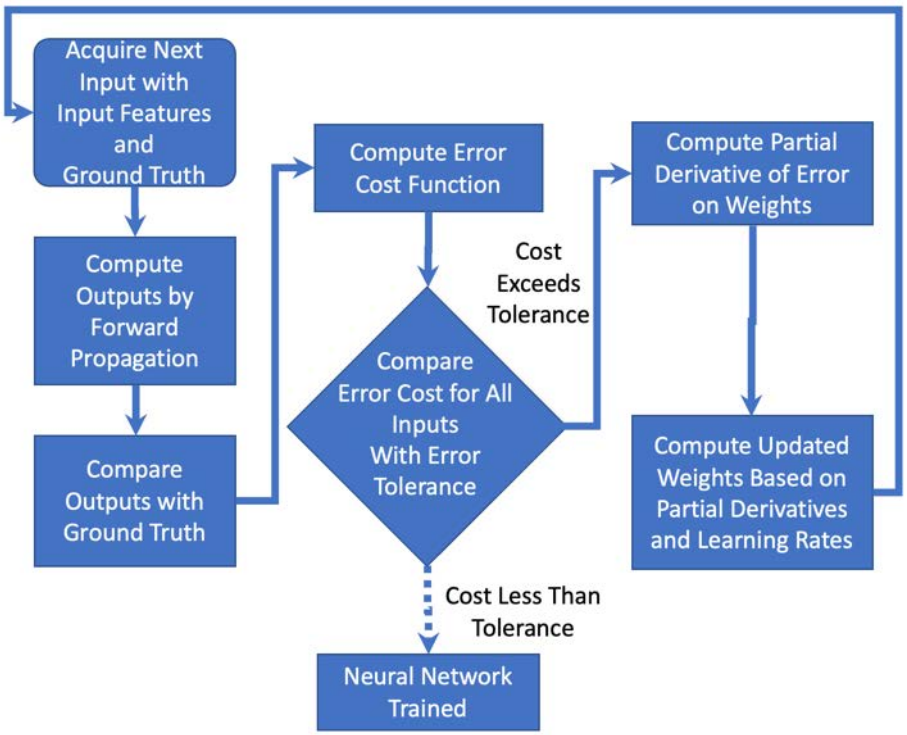
Neural Networks Training

Forward vs Backward Propagation

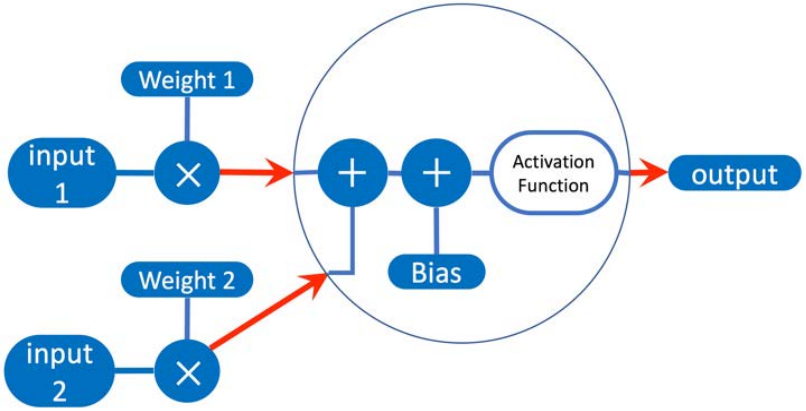
Slides are partially adapted from notes by William Kaiser, Vincent Cheung, and Kevin Cannons

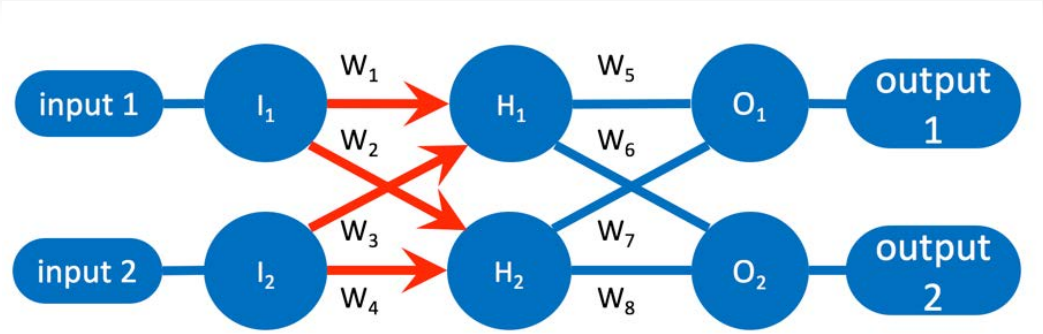
Ira A. Fulton Schools of Engineering
Arizona State University

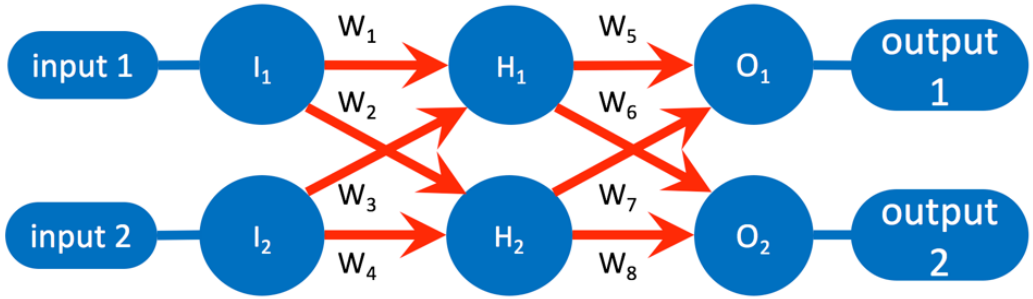
Recall: Gradient Descent Training Algorithm

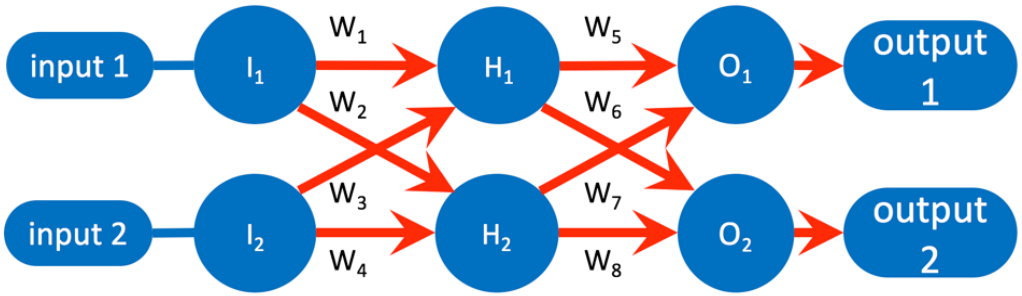


Compute Outputs By Forward Propagation





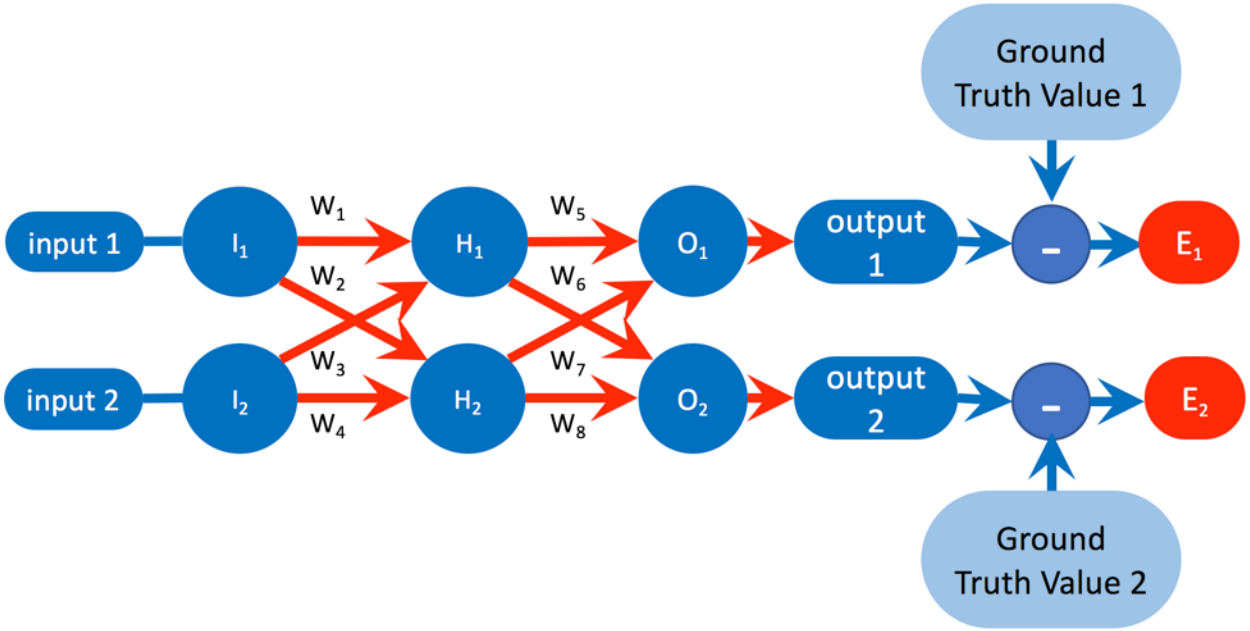




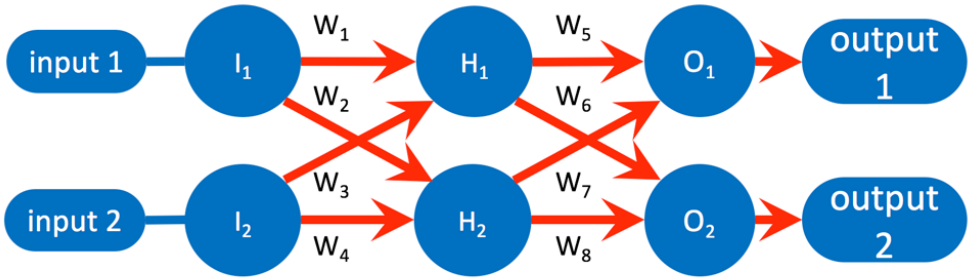
Compute Error Cost Function

$$E(W,b) = \sum_{i=1}^{M} \frac{1}{2}[Ground\ Truth_i - Output_i]^2$$

For example, M = 2, E = E_1 + E_2.



Example



Given

- Bias at H_1 and H_2: $b_1 = 0.25$
- Bias at O_1 and O_2: $b_2 = 0.35$
- Weights: $w_1 = 0.1$, $w_2 = 0.2$, $w_3 = 0.3$, $w_4 = 0.4$
- Weights: $w_5 = 0.5$, $w_6 = 0.6$, $w_7 = 0.7$, $w_8 = 0.8$
- Sigmoid activation function for H_1, H_2 and O_1, O_2 : $f(y) = \frac{1}{1+e^{-y}}$
- Inputs: input1 = 0.1, input2 = 0.5

What is the value of output1 and output2?

If the ground truth at output1 and output2 is 0.05 and 0.95, what is the value of the error cost function?

Answer

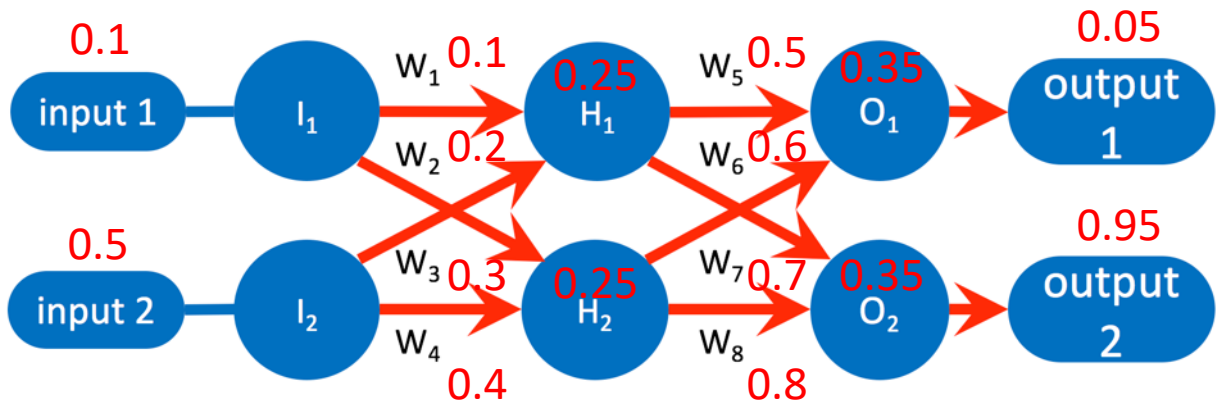
$$output_{h1} = f(0.1 * 0.1 + 0.5 * 0.3 + 0.25) = \frac{1}{1+e^{-0.41}} = 0.6$$

$$output_{h2} = f(0.1 * 0.2 + 0.5 * 0.4 + 0.25) = \frac{1}{1+e^{-0.47}} = 0.62$$

$$output_{o1} = f(0.6 * 0.5 + 0.62 * 0.7 + 0.35) = \frac{1}{1+e^{-1.084}} = 0.75$$

$$output_{o2} = f(0.6 * 0.6 + 0.62 * 0.8 + 0.35) = \frac{1}{1+e^{-1.2}} = 0.77$$

$$E = E_1 + E_2 = \frac{1}{2}(0.05 - 0.75)^2 + \frac{1}{2}(0.95 - 0.77)^2 = 0.26$$



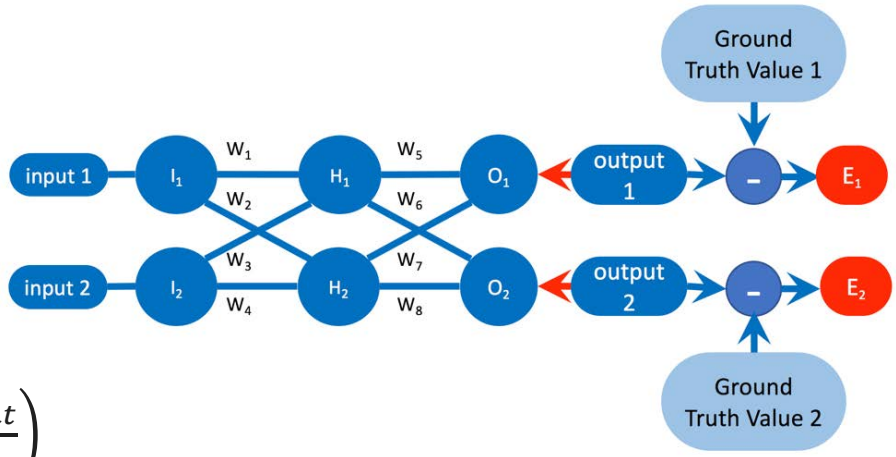
Compute Partial Derivative of Error on Weights

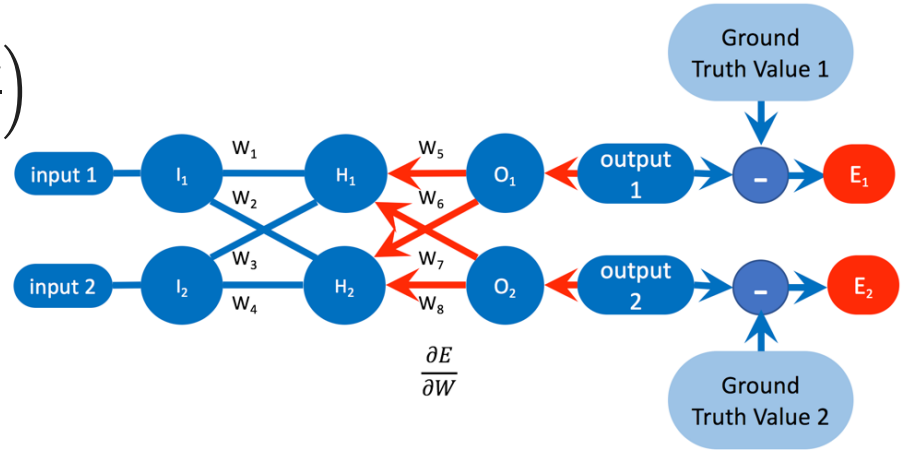
- For simplicity, assume biases are given, need to update weights only
- To use gradient descent algorithm, need to compute the partial derivative of error with respect to each weight
  - $\left(\frac{\partial E}{\partial W_1}\right), \left(\frac{\partial E}{\partial W_2}\right), \left(\frac{\partial E}{\partial W_3}\right), \left(\frac{\partial E}{\partial W_4}\right), \left(\frac{\partial E}{\partial W_5}\right), \left(\frac{\partial E}{\partial W_6}\right), \left(\frac{\partial E}{\partial W_7}\right), \left(\frac{\partial E}{\partial W_8}\right)$

Compute Partial Derivatives

To calculate $\left(\frac{\partial E}{\partial W_5}\right), \left(\frac{\partial E}{\partial W_6}\right), \left(\frac{\partial E}{\partial W_7}\right), \left(\frac{\partial E}{\partial W_8}\right)$, need to use chain rule to calculate

- $\left(\frac{\partial E}{\partial O_{1Output}}\right), \left(\frac{\partial E}{\partial O_{2Output}}\right)$
- $\left(\frac{\partial O_{1Output}}{\partial O_{1Input}}\right), \left(\frac{\partial O_{2Output}}{\partial O_{2Input}}\right)$
- $\left(\frac{\partial O_{1Input}}{\partial W_5}\right), \left(\frac{\partial O_{1Input}}{\partial W_7}\right), \left(\frac{\partial O_{2Input}}{\partial W_6}\right), \left(\frac{\partial O_{2Input}}{\partial W_8}\right)$
- For example:
- $\left(\frac{\partial E}{\partial W_5}\right) = \left(\frac{\partial E}{\partial O_{1Output}}\right)\left(\frac{\partial O_{1Output}}{\partial O_{1Input}}\right)\left(\frac{\partial O_{1Input}}{\partial W_5}\right)$
- $\left(\frac{\partial E}{\partial O_{1Output}}\right) = \left(\frac{\partial E_1}{\partial O_{1Output}}\right) + \left(\frac{\partial E_2}{\partial O_{1Output}}\right)$

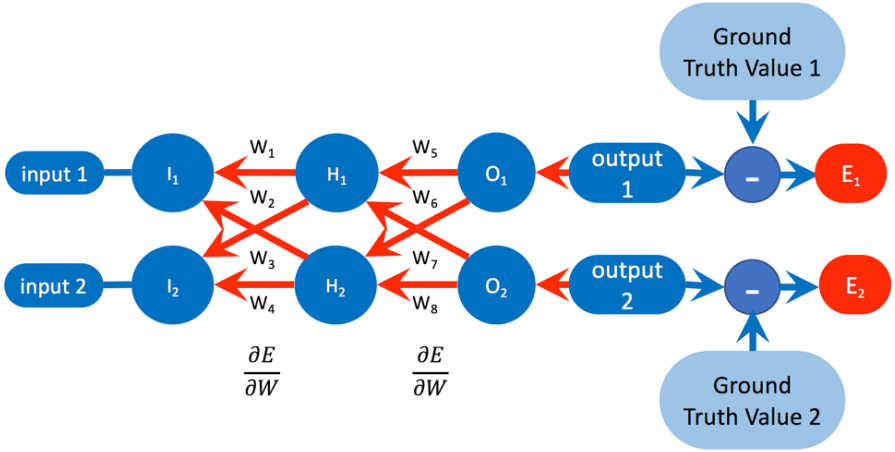




Compute Partial Derivatives (cont'd)

To calculate $\left(\frac{\partial E}{\partial W_1}\right), \left(\frac{\partial E}{\partial W_2}\right), \left(\frac{\partial E}{\partial W_3}\right), \left(\frac{\partial E}{\partial W_4}\right)$, need to use chain rule to calculate

- $\left(\frac{\partial E}{\partial H_{1_{Output}}}\right), \left(\frac{\partial E}{\partial H_{2_{Output}}}\right)$ (need to further apply chain rule)
- $\left(\frac{\partial H_{1_{Output}}}{\partial H_{1_{Input}}}\right), \left(\frac{\partial H_{2_{Output}}}{\partial H_{2_{Input}}}\right)$
- $\left(\frac{\partial H_{1_{Input}}}{\partial W_1}\right), \left(\frac{\partial H_{1_{Input}}}{\partial W_3}\right), \left(\frac{\partial H_{2_{Input}}}{\partial W_2}\right), \left(\frac{\partial H_{2_{Input}}}{\partial W_4}\right)$



Backpropagation

Calculation of the derivatives flows backwards through the network, hence the name, backpropagation.

Partial derivatives of the error with respect to the weights:

- Output neurons:
  - j = output neuron, i = neuron in last hidden, f(·) activation function
  - net_j = output of neuron j, $output_i = input_j$
  - let: $\delta_j = f'(net_j)\ (target_j - output_j)$
  - $\partial E/\partial w_{ji} = -output_i\ \delta_j$
- Hidden layer neurons:
  - j = hidden neuron, i = neuron in previous layer, k = neuron in next layer, f(·) activation function
  - net_j = output of neuron j, $output_i = input_j$
  - let: $\delta_j = f'(net_j)\ \Sigma(\delta_k w_{kj})$
  - $\partial E/\partial w_{ji} = -output_i\ \delta_j$

Compute Updated Weights

- **The partial derivatives point in the direction of the maximum increase of the error function.**

- **A small step (learning rate) in the opposite direction will result in the maximum decrease of the (local) error function:**
  - Gradient descent: $w_{new} = w_{old} – \alpha\ \partial E/\partial w_{old,}$ α is the learning rate

- **Momentum: extension to the gradient descent algorithm**
  - Applies smoothed averaging to the change in weights:
    - $\Delta_{new} = \beta\Delta_{old} - \alpha\ \partial E/\partial w_{old}$, β is the momentum coefficient
    - $w_{new} = w_{old} + \Delta_{new}$
  - Acts as a low-pass filter by reducing rapid fluctuations
  - Tends to aid convergence

Backpropagation Derivation and Example

- **For detailed derivation, see the Neural Network Execution and Training handout**
- **For a numerical example:**
  - A Step by Step Backpropagation Example

When to stop?

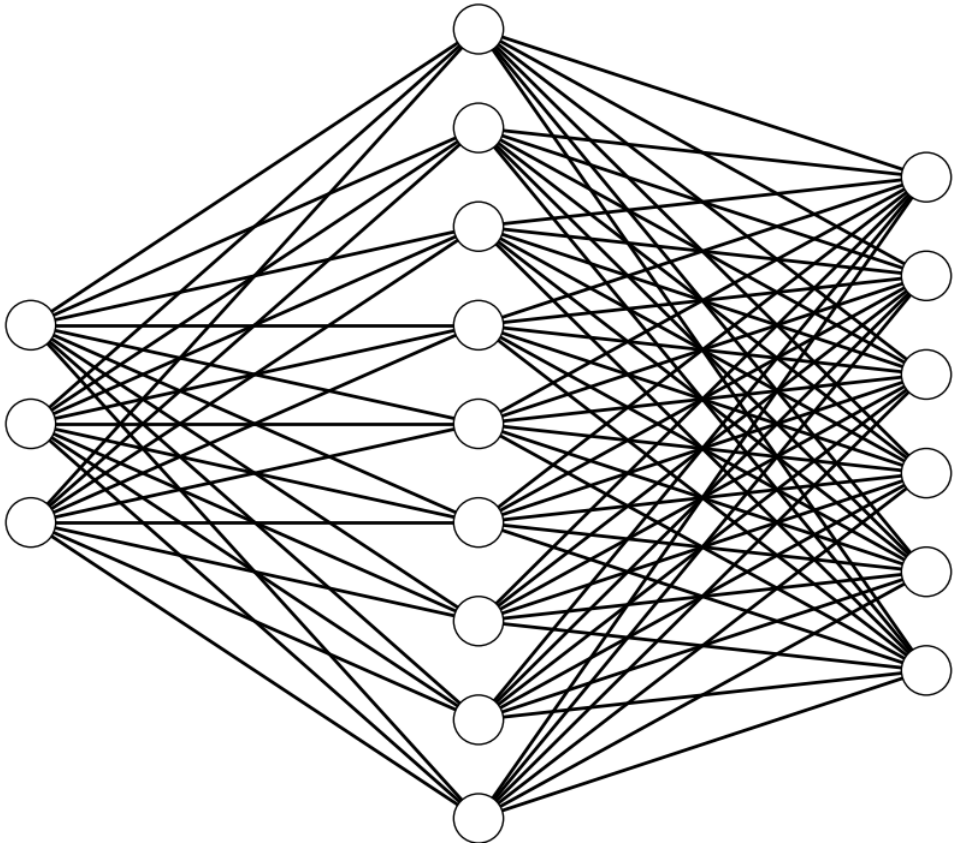
Commonly used stopping criteria for training

- Error cost function is below a threshold.
- The maximum number of iterations has passed.
- There is little change in error cost function.
- Etc.

Example

For the shown neural network topology,

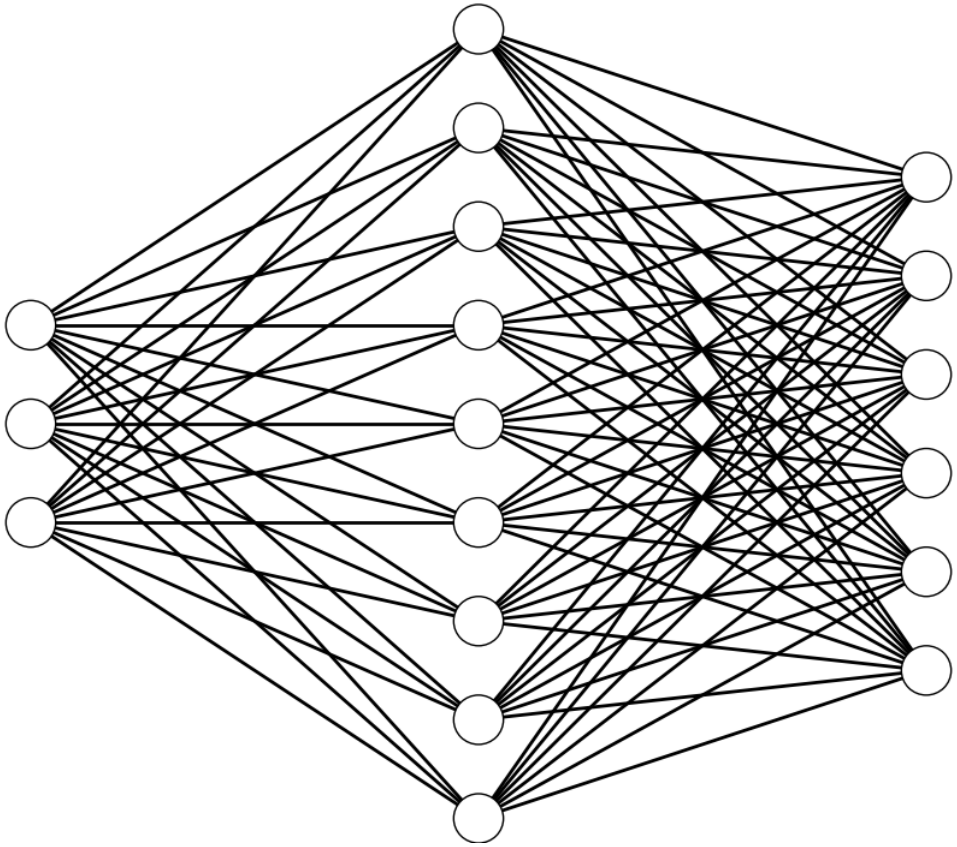
- How many inputs?
- How many hidden layers? How many neurons for each?
- How many outputs?
- How many total weight values?
- How many total bias values?
- What is the number of trainable parameters?



Answer

For the shown neural network topology,

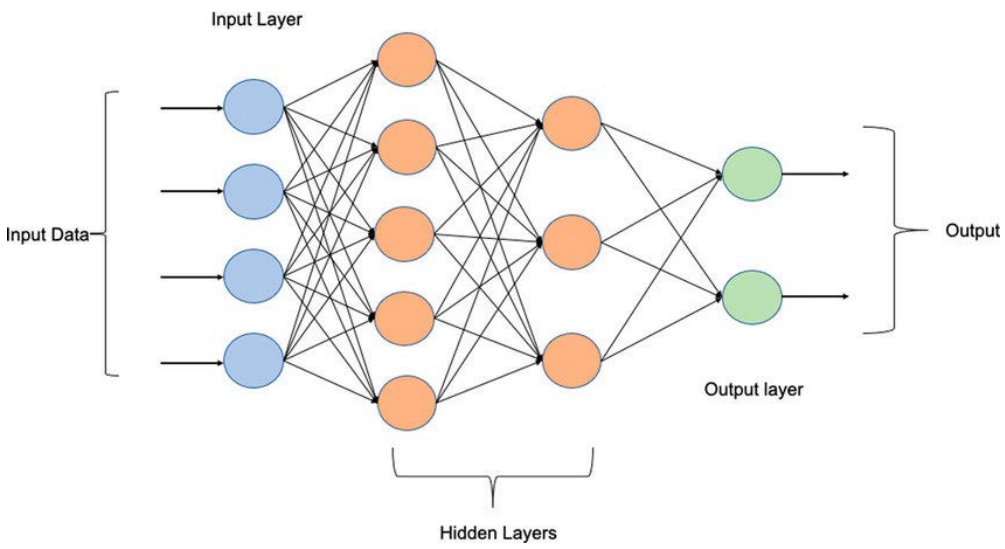
- How many inputs? 3
- How many hidden layers? How many neurons for each? 1, 9
- How many outputs? 6
- How many total weight values?
  - 3*9 + 9*6 = 81
- How many total bias values?
  - 9 + 6 = 15
- What is the number of trainable parameters?
  - 81 + 15 = 96



Question

For the shown neural network topology,

- How many inputs?
- How many hidden layers? How many neurons for each?
- How many outputs?
- How many total weight values?
- How many total bias values?
- What is the number of trainable parameters?



Ira A. Fulton Schools of Engineering

Arizona State University